

Distributed Communication Project Report

Video link: <https://youtu.be/oZDB6VIXQs?si=7NS9MMFOONSvtVY8>

Design Description:

This project implements Anycast and Multicast communication patterns using Docker containerization. The **anycast implementation** uses TCP with three servers listening on port 5000, where Docker's DNS round-robin routes client connections to different servers demonstrating **load balancing**—each connection goes to only one server, but multiple clients distribute across all servers. The server.py implements `socket.socket(AF_INET, SOCK_STREAM)` for **reliable TCP connections** with `bind()`, `listen()`, and `accept()` creating connection-oriented endpoints that guarantee delivery through acknowledgments. The client.py establishes connections using `connect()`, demonstrating TCP's stateful three-way handshake (SYN, SYN-ACK, ACK).

The **multicast implementation** uses UDP for **one-to-many communication** with two senders broadcasting to multicast group 224.1.1.1:5007, where multiple receivers simultaneously receive messages like a radio broadcast. The multicast_sender.py implements `socket.socket(AF_INET, SOCK_DGRAM)` with `IP_MULTICAST_TTL` to control packet propagation, sending three message types: plain text, JSON sensor data, and binary data. The multicast_receiver.py joins the multicast group using `struct.pack()` and `IP_ADD_MEMBERSHIP`, implementing a `--duration` argument for timed group departure, demonstrating that **receivers can join/leave independently** without affecting others. Docker Compose orchestrates seven containers using bridge networks for DNS resolution and multicast packet forwarding.

Why UDP Multicast is Not Reliable

UDP multicast operates as **best-effort delivery without reliability guarantees**. Unlike TCP's acknowledgment system, UDP sends packets fire-and-forget with **no confirmation mechanism**—senders never know if packets reached destinations, and receivers cannot request retransmissions. UDP lacks flow control and congestion control, so network congestion causes packet drops without notification.

```
PS C:\Users\melod\CECS-327\Project-2-Distr-Comm\multicast> docker exec multicast_receiver1 tcpdump -i eth0 udp port 5007 -n -v
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
22:50:55.920655 IP (tos 0x0, ttl 2, id 54567, offset 0, flags [DF], proto UDP (17), length 60)
 172.18.0.2.49455 > 224.1.1.1.5007: UDP, length 32
22:50:55.943082 IP (tos 0x0, ttl 2, id 7751, offset 0, flags [DF], proto UDP (17), length 60)
 172.18.0.3.38540 > 224.1.1.1.5007: UDP, length 32
22:50:57.923127 IP (tos 0x0, ttl 2, id 54689, offset 0, flags [DF], proto UDP (17), length 123)
 172.18.0.2.49455 > 224.1.1.1.5007: UDP, length 95
22:50:57.945630 IP (tos 0x0, ttl 2, id 8251, offset 0, flags [DF], proto UDP (17), length 123)
 172.18.0.3.38540 > 224.1.1.1.5007: UDP, length 95
22:51:18.172003 IP (tos 0x0, ttl 2, id 54792, offset 0, flags [DF], proto UDP (17), length 38)
 172.18.0.2.49455 > 224.1.1.1.5007: UDP, length 10
22:51:18.194574 IP (tos 0x0, ttl 2, id 8698, offset 0, flags [DF], proto UDP (17), length 38)
 172.18.0.3.38540 > 224.1.1.1.5007: UDP, length 10
22:51:20.174491 IP (tos 0x0, ttl 2, id 55204, offset 0, flags [DF], proto UDP (17), length 60)
 172.18.0.2.49455 > 224.1.1.1.5007: UDP, length 32
22:51:20.197067 IP (tos 0x0, ttl 2, id 8775, offset 0, flags [DF], proto UDP (17), length 60)
 172.18.0.3.38540 > 224.1.1.1.5007: UDP, length 32
```

Our tcpdump captures show UDP packets transmitted without acknowledgement responses. In Docker's local network we observed no loss, but in real networks, packets frequently drop due to congestion or unreliable links. The protocol provides **no ordering guarantees**—packets sent sequentially (0, 1, 2) might arrive out of order (2, 0, 1). Our receivers decoded text, JSON, and binary messages, but dropped packets would simply disappear with no error indication.

How Group Join/Leave Affects Packet Reception

Multicast group membership operates like a subscription—**only receivers who explicitly joined receive packets**. The `IP_ADD_MEMBERSHIP` socket option tells the kernel to forward packets for multicast address 224.1.1.1 to the application's socket. Before joining, receivers ignore all multicast traffic. After leaving, packet delivery stops immediately.

```
Container multicast_receiver1 Created 0.2s
Attaching to multicast_receiver1, multicast_receiver2, multicast_sender1, multicast_sender2
multicast_sender1 | sender1 sending to 224.1.1.1:5007
multicast_sender1 | Sent text: Multicast message 0 from sender1
multicast_sender2 | sender2 sending to 224.1.1.1:5007
multicast_sender2 | Sent text: Multicast message 0 from sender2
multicast_receiver2 | Joined multicast group 224.1.1.1:5007
multicast_receiver1 | Joined multicast group 224.1.1.1:5007
multicast_receiver2 | Received JSON: {'sender': 'sender1', 'sensor': 'temperature', 'value': 26.36, 'timestamp': 1759964273.4472995} from ('172.18.0.2', 43801)
```

```
multicast_receiver1 | Leaving multicast group
multicast_receiver1 exited with code 0
multicast_receiver2 | Received binary: f5bc71e0bc51a0077df4 from ('172.18.0.2', 43801)
multicast_sender1 | Sent binary: f5bc71e0bc51a0077df4
```

```
multicast_receiver2 | Received binary: 52e2e9a3810088d10c21 from ('172.18.0.3', 33543)
multicast_sender2 | Sent binary: 52e2e9a3810088d10c21

multicast_sender1 | Sent text: Multicast message 3 from sender1der1 from ('172.18.0.2', 43801)

multicast_receiver2 | Received text: Multicast message 3 from sender2 from ('172.18.0.3', 33543)
multicast_sender2 | Sent text: Multicast message 3 from sender2
multicast_receiver2 | Leaving multicast group
multicast_receiver2 exited with code 0
```

Our logs show both receivers printing "Joined multicast group 224.1.1.1:5007" then receiving messages from both sender1 (172.18.0.3) and sender2 (172.18.0.2). Receiver1 ran for 120 seconds then printed "Leaving multicast group" and exited, while receiver2 continued until 150 seconds. During the 30-second gap, receiver2 kept receiving messages, proving **one receiver's departure doesn't affect others**. Late joiners miss previous messages—multicast has no replay mechanism. **Senders remain unaware** of receiver count, broadcasting continuously whether zero or many listeners exist.

Single-Source vs Multi-Source Multicast Differences

Single-source multicast has all packets from one sender IP with predictable timing.

Multi-source multicast has packets interleaving from multiple sender IPs with independent timing.

```

PS C:\Users\melod\CECS-327\Project-2-Distr-Comm\multicast> docker exec multicast_receiver1 tcpdump -i eth0 udp
port 5007 -n -v
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
22:50:55.920655 IP (tos 0x0, ttl 2, id 54567, offset 0, flags [DF], proto UDP (17), length 60)
    172.18.0.2.49455 > 224.1.1.1.5007: UDP, length 32
22:50:55.943082 IP (tos 0x0, ttl 2, id 7751, offset 0, flags [DF], proto UDP (17), length 60)
    172.18.0.3.38540 > 224.1.1.1.5007: UDP, length 32
22:50:57.923127 IP (tos 0x0, ttl 2, id 54689, offset 0, flags [DF], proto UDP (17), length 123)
    172.18.0.2.49455 > 224.1.1.1.5007: UDP, length 95
22:50:57.945630 IP (tos 0x0, ttl 2, id 8251, offset 0, flags [DF], proto UDP (17), length 123)
    172.18.0.3.38540 > 224.1.1.1.5007: UDP, length 95
22:51:18.172003 IP (tos 0x0, ttl 2, id 54792, offset 0, flags [DF], proto UDP (17), length 38)
    172.18.0.2.49455 > 224.1.1.1.5007: UDP, length 10
22:51:18.194574 IP (tos 0x0, ttl 2, id 8698, offset 0, flags [DF], proto UDP (17), length 38)
    172.18.0.3.38540 > 224.1.1.1.5007: UDP, length 10
22:51:20.174491 IP (tos 0x0, ttl 2, id 55204, offset 0, flags [DF], proto UDP (17), length 60)
    172.18.0.2.49455 > 224.1.1.1.5007: UDP, length 32
22:51:20.197067 IP (tos 0x0, ttl 2, id 8775, offset 0, flags [DF], proto UDP (17), length 60)
    172.18.0.3.38540 > 224.1.1.1.5007: UDP, length 32

```

```

multicast_sender2 | Sent JSON: {"sender": "sender2", "sensor": "temperature", "value": 28.02, "timestamp":
1759964092.923671}

multicast_receiver1 | Received JSON: {'sender': 'sender1', 'sensor': 'temperature', 'value': 25.59, 'timestam
p': 1759964092.957679} from ('172.18.0.3', 48976)
multicast_sender1 | Sent JSON: {"sender": "sender1", "sensor": "temperature", "value": 25.59, "timestamp":
1759964092.957679}

multicast_receiver2 | Received JSON: {'sender': 'sender1', 'sensor': 'temperature', 'value': 25.59, 'timestam
p': 1759964092.957679} from ('172.18.0.3', 48976)
multicast_sender2 | Sent binary: 083cb2ee30c765a3454f

multicast_receiver1 | Received binary: 083cb2ee30c765a3454f from ('172.18.0.2', 37255)

multicast_receiver2 | Received binary: 083cb2ee30c765a3454f from ('172.18.0.2', 37255)

multicast_receiver1 | Received binary: ac4d5f8470eed5063de6 from ('172.18.0.3', 48976)
multicast_sender1 | Sent binary: ac4d5f8470eed5063de6

multicast_receiver2 | Received binary: ac4d5f8470eed5063de6 from ('172.18.0.3', 48976)
multicast_receiver1 | Received text: Multicast message 3 from sender2 from ('172.18.0.2', 37255)

```

Our tcpdump shows packets from two source IPs (172.18.0.3 and 172.18.0.2) transmitting to the **same destination multicast address** (224.1.1.1:5007) but from different ports. Receiver logs show **interleaved messages** with sender1 and sender2 transmitting nearly simultaneously (timestamps differ by 0.000138 seconds). Both receivers simultaneously receive messages from **both senders**, handling multiple data types—JSON sensor data and binary data. **Receivers cannot filter** at kernel level—once joined to 224.1.1.1, the socket receives packets from ALL senders. Our code inspects the **address** tuple from `recvfrom()` to identify sender IPs, but this happens after packet delivery.

Screenshots of Successful Runs:

Anycast Load Balancing (demonstrates round-robin distribution):

```

PS C:\Users\melod\CECS-327\Project-2-Distr-Comm\anycast> docker exec anycast_client python client.py
Received: Hello from server3
PS C:\Users\melod\CECS-327\Project-2-Distr-Comm\anycast> docker exec anycast_client python client.py
Received: Hello from server1
PS C:\Users\melod\CECS-327\Project-2-Distr-Comm\anycast> docker exec anycast_client python client.py
Received: Hello from server1
PS C:\Users\melod\CECS-327\Project-2-Distr-Comm\anycast> docker exec anycast_client python client.py
Received: Hello from server3
PS C:\Users\melod\CECS-327\Project-2-Distr-Comm\anycast> docker exec anycast_client python client.py
Received: Hello from server3
PS C:\Users\melod\CECS-327\Project-2-Distr-Comm\anycast> docker exec anycast_client python client.py
Received: Hello from server2
PS C:\Users\melod\CECS-327\Project-2-Distr-Comm\anycast> docker exec anycast_client python client.py
Received: Hello from server2
PS C:\Users\melod\CECS-327\Project-2-Distr-Comm\anycast> docker exec anycast_client python client.py
Received: Hello from server2

```

Anycast tcpdump (TCP 3-way handshake and reliable delivery):

```

22:34:55.838109 IP 172.18.0.5.58860 > 172.18.0.3.5000: Flags [S], seq 2880226401, win 64240, options [mss 1460
,sackOK,TS val 804337561 ecr 0,nop,wscale 7], length 0
22:34:55.838140 IP 172.18.0.3.5000 > 172.18.0.5.58860: Flags [S.], seq 3200949366, ack 2880226402, win 65160,
options [mss 1460,sackOK,TS val 3501676442 ecr 804337561,nop,wscale 7], length 0
22:34:55.838178 IP 172.18.0.5.58860 > 172.18.0.3.5000: Flags [.], ack 1, win 502, options [nop,nop,TS val 8043
37561 ecr 3501676442], length 0
22:34:55.838561 IP 172.18.0.3.5000 > 172.18.0.5.58860: Flags [P.], seq 1:20, ack 1, win 510, options [nop,nop,
TS val 3501676442 ecr 804337561], length 19
22:34:55.838646 IP 172.18.0.5.58860 > 172.18.0.3.5000: Flags [.], ack 20, win 502, options [nop,nop,TS val 804
337561 ecr 3501676442], length 0
22:34:55.838737 IP 172.18.0.5.58860 > 172.18.0.3.5000: Flags [F.], seq 1, ack 20, win 502, options [nop,nop,TS
val 804337561 ecr 3501676442], length 0
22:34:55.838744 IP 172.18.0.3.5000 > 172.18.0.5.58860: Flags [F.], seq 20, ack 1, win 510, options [nop,nop,TS
val 3501676442 ecr 804337561], length 0
22:34:55.838786 IP 172.18.0.5.58860 > 172.18.0.3.5000: Flags [.], ack 21, win 502, options [nop,nop,TS val 804
337561 ecr 3501676442], length 0
22:34:55.838800 IP 172.18.0.3.5000 > 172.18.0.5.58860: Flags [.], ack 2, win 510, options [nop,n]

```

TCP flags: [S] (syn), [S.] (syn-ack), [.] (ack), [P.] (data), [F.] (fin)

Multicast Multi-Source Reception:

```

multicast_receiver1 | Joined multicast group 224.1.1.1:5007
multicast_receiver2 | Joined multicast group 224.1.1.1:5007
multicast_receiver2 | Received JSON: {'sender': 'sender1', 'sensor': 'temperature', 'value': 28.2, 'timestamp
': 1759962979.7531772} from ('172.18.0.2', 45828)
multicast_sender1 | Sent JSON: {"sender": "sender1", "sensor": "temperature", "value": 28.2, "timestamp": 1
759962979.7531772}
multicast_receiver1 | Received JSON: {'sender': 'sender1', 'sensor': 'temperature', 'value': 28.2, 'timestamp
': 1759962979.7531772} from ('172.18.0.2', 45828)
multicast_sender2 | Sent JSON: {"sender": "sender2", "sensor": "temperature", "value": 29.68, "timestamp":
1759962979.7865345}
multicast_receiver1 | Received JSON: {'sender': 'sender2', 'sensor': 'temperature', 'value': 29.68, 'timestamp
p': 1759962979.7865345} from ('172.18.0.3', 49880)
multicast_receiver2 | Received JSON: {'sender': 'sender2', 'sensor': 'temperature', 'value': 29.68, 'timestamp
p': 1759962979.7865345} from ('172.18.0.3', 49880)

```

Challenges and Solutions

- **Python Output Buffering:** Logs weren't appearing. Added PYTHONUNBUFFERED=1 environment variable and flush=True to print statements.
- **Anycast DNS:** Client connected to localhost instead of Docker DNS. Changed to hostname "server" with network aliases in docker-compose.yml.
- **Multicast Membership:** Used struct.pack() to properly format IP_ADD_MEMBERSHIP socket option.

- **tcpdump on Short-Lived Containers:** Captured on sender1 instead of receiver1, increased receiver durations to 120/150 seconds.

Individual Contributions:

Melody:

- Implemented server.py with TCP sockets and multicast_sender.py with UDP sockets
- Developed three message types (text, JSON, binary) and multicast group membership code
- Captured tcpdump traffic and recorded demonstration video

Andrea:

- Created client.py with TCP connections and multicast_receiver.py with --duration parameter
- Implemented docker-compose.yml files with network configurations
- Wrote README.md and tested container orchestration

References

Docker Networking Documentation

Python Socket Programming Documentation

CECS 327 Course Materials